
CHAPTER 3

Lab 3: RISC-V-lite

3.1 Architecture definition

3.1.1 Control registers

The design makes use of two control registers.

- CONTROL
 - bit 0: START (When this bit is set to 1 by software, the conv1d accelerator should start)
 - bit 1: ACCUMULATE (When 1: read/write outputs, accumulate to existing partial sum) (When 0: overwrite, no accumulation is done)
- STATUS
 - bit 0: RUNNING (This bit is set to 1 when the accelerator is running)
 - bit 1: DONE (This bit is set to 1 when the accelerator has finished its computation)

The register bit positions and offsets are auto-generated from the (`control_reg.hjson`) via `make gen-registers`. This produces both the RTL register interface and a C header (`conv1d_control_reg.h`) with `#define` constants for each register offset and bit position.

The START bit is set to 1 by the testbench after the input and weight/kernel vectors are uploaded to the scratchpad. This makes the FSM leave IDLE and compute the first tile. Software clears START after completion.

The ACCUMULATE signal identifies the first input-channel tile. When ACCUMULATE = 0, no previous partial sums are read. This bypass saves 48 cycles per overwrite run (2 cycles for each of the 24 physical outputs). Across 44 overwrite runs, the total reduction is

$$48 \times 44 = 2,112 \text{ cycles.}$$

Usage of the STATUS register:

The RUNNING bit indicates whether the accelerator is currently running. This means that the accelerator is no longer in the IDLE or DONE state, but is actively computing the convolutions.

The DONE bit is instead used as a flag to signal that the accelerator has completed its computation. It is set to “1” when the FSM reaches the DONE state. This allows the testbench either to upload new vectors or to terminate the program.

3.1.2 Finite State Machine (FSM)

The Finite State Machine (FSM) is implemented in the `conv1d.sv` module and manages the entire execution flow of the accelerator: weight loading, window initialization, convolution computation, accumulation, and output write-back.

The states are defined using a 4-bit enum:

```

1 // FSM states
2 typedef enum logic [3:0] {
3     IDLE,
4     READ_PREV_OUT,      // Read previous output for accumulation
5     WAIT_PREV_OUT,      // Wait for previous output data
6     FETCH_INPUTS,       // Fetch 4 inputs from SRAM (4 cycles)
7     FETCH_WEIGHTS,      // Fetch 4 weights from SRAM (4 cycles)
8     WAIT_CAPTURE,       // Wait for last weight to be captured
9     COMPUTE,            // Compute 4 MACs + accumulate
10    WRITE_OUTPUT,        // Write result to SRAM
11    NEXT_KERNEL,         // Move to next kernel position
12    NEXT_OUTPUT,         // Move to next output
13    DONE
14 } state_t;
15 state_t state_q, state_d;
```

Listing 3.1: FSM state definition in `conv1d.sv`

Below is the complete list of states, along with a brief description of their purpose and main operations:

IDLE Initial idle state. The accelerator waits for the software to set `start_i = 1` (START bit in the control registers). The signal `ext_mem_gnt = 1` grants the host access to the SRAM. Transition to `READ_PREV_OUT` when `start_i = 1` & `accumulate_i = 1`, otherwise skip to `FETCH_INPUTS`.

READ_PREV_OUT If `accumulate_i = 1`, initiates read of the current output value from SRAM. Issues read request to output address. Transitions to `WAIT_PREV_OUT`.

WAIT_PREV_OUT Waits for SRAM read latency (1 cycle). Captures previous output value from `rdata`. Adds it later in `COMPUTE`. Transitions to `FETCH_INPUTS`.

FETCH_INPUTS Fetches 4 input samples (4 x 32-bit word input value) from SRAM for the current tile. Issues sequential reads. Stores in input buffer `in_pipe_q`. Transitions to `FETCH_WEIGHTS`.

FETCH_WEIGHTS Fetches 4 weights samples (for the 4 input channels) from SRAM for the current output channel tile and kernel position. Issues reads, stores in weight buffer `wt_pipe_q`. Transitions to `WAIT_CAPTURE`.

WAIT_CAPTURE Waits for the last weight read to complete. Ensures all data ready for MAC. Transitions to `COMPUTE`.

COMPUTE Performs 4 parallel multiplication on current input/weight pair. Uses adder tree to sum products (3 adders total). Accumulates result in register `accum_q` (fourth adder) (The representation is done in figure 3.1). Transitions to `NEXT_KERNEL`.

NEXT_KERNEL Kernel counter. Loops back to `FETCH_INPUTS` until 5 input/weight pair are computed to make a partial sum, otherwise, transition to `WRITE_OUTPUT`.

WRITE_OUTPUT Writes accumulated result (or accumulated + previous) to SRAM at output address. Issues write transaction. Transitions to `NEXT_OUTPUT`.

NEXT_OUTPUT Output position counter. If tile complete, advances to next tile, otherwise loops back to `FETCH_INPUTS` if `accumulate_i = 0`, or loops back to `READ_PREV_OUT` if `accumulate_i = 1`. If all tiles done, transitions to `DONE`.

DONE Final state. Sets `done_o = 1` (DONE bit in the control registers), releases the SRAM (`ext_mem_gnt = 1`). Returns to `IDLE` when the software resets the start signal.

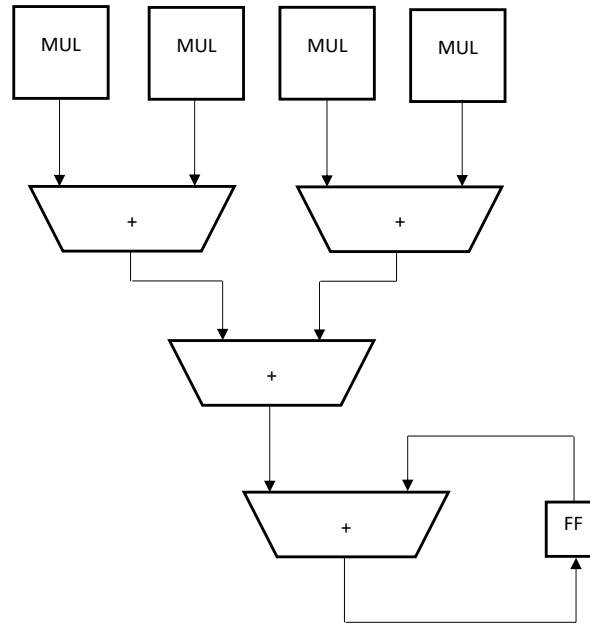


Figure 3.1: Adder and multiplier arrangement

Even though the multipliers are arranged horizontally, they operate on input vector values belonging to four different channels (vertically, if one considers the representation in Figure 3.3). The first three adders are used to compute the sum of the products, while the last one (the fourth) is dedicated to accumulation. In fact, the depicted flip-flop represents an abstraction of one of the accelerator's output vectors.

Initially, a more linear structure for the adder and multiplier network was considered:

```
1 assign add_tree_out = (((mul_result[0] + mul_result[1]) + mul_result[2]) +
    mul_result[3]); // 1, 2, 3 adder
```

Listing 3.2: First structure of adders and multiplier

However, with this approach the critical path was equal to 1 multiplier plus 4 adders. By using a tree-based structure, it was possible to shorten the critical path, which in this case becomes 1 multiplier plus 3 adders. The code is shown below:

```
1 assign add_tree_out = (mul_result[0] + mul_result[1]) + (mul_result[2] +
    mul_result[3]); // 1, 2, 3 adder
```

Listing 3.3: Tree structure of adders and multiplier

Figure 3.2 shows a flow chart representing the connections between the states of the FSM:

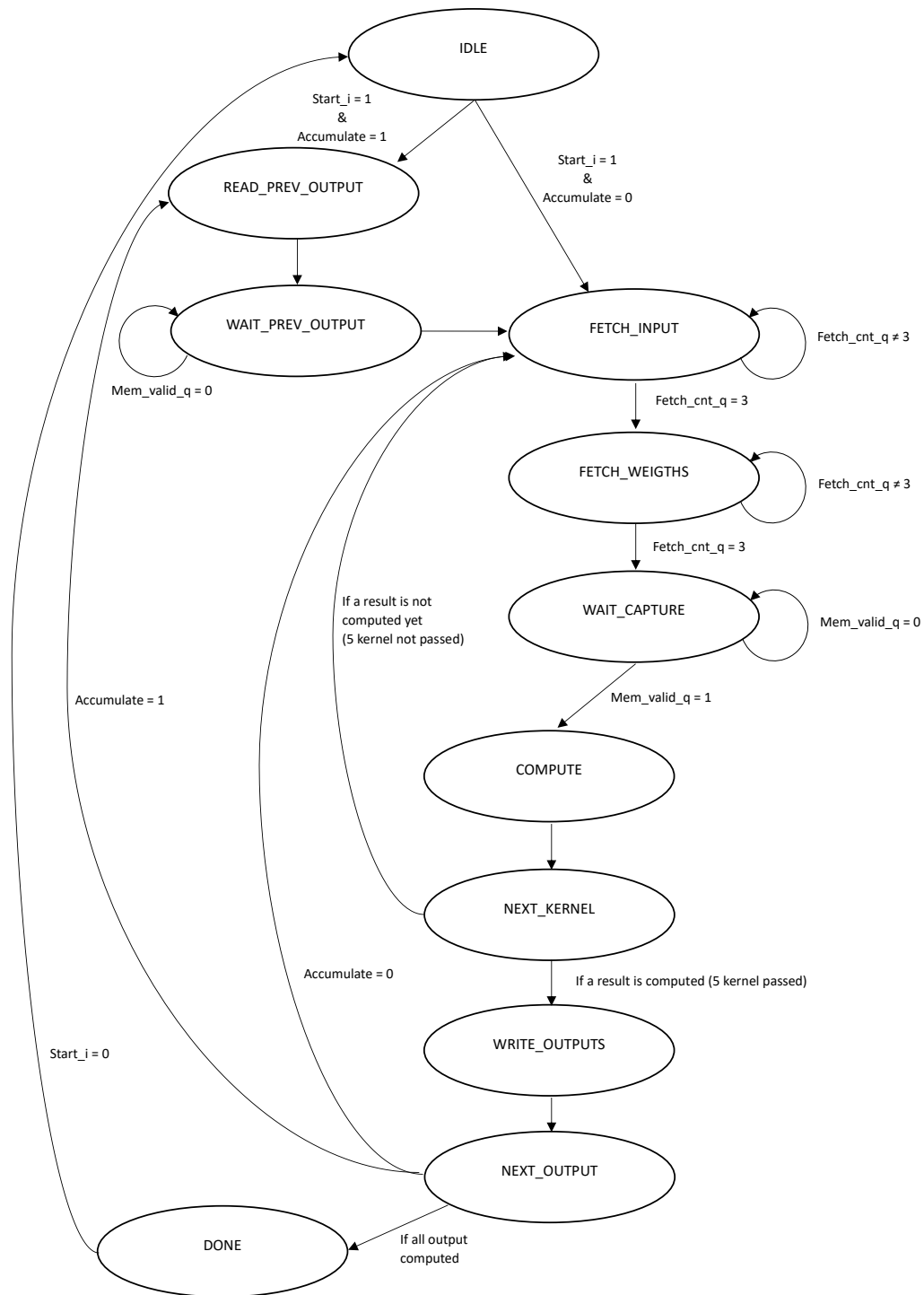


Figure 3.2: Flow chart FSM

3.1.3 Tiling management

The internal scratchpad memory of the accelerator has a capacity of only 128 32-bit words (512 bytes in total), while the overall data required for the convolution largely exceeds this limit:

- Input: $16 \text{ channels} \times 128 \text{ samples} = 2048 \text{ words}$
- Weights: $8 \text{ output channels} \times 16 \text{ input channels} \times 5 \text{ kernel} = 640 \text{ words}$
- Output: $8 \text{ channels} \times 124 \text{ samples} = 992 \text{ words}$
- Total: 3680 words

To address this issue, a 3D tiling approach was adopted over the following dimensions: input channels (C_{in}), output channels (C_{out}), and temporal dimension (T, the output sequence length). This allows the hardware to process one tile at a time, keeping all the required data within the available 128 words.

Tile Parameters:

- **TILE_CIN = 4:** Input channels per tile (maximum parallelism of the 4 hardware multipliers).
- **TILE_COUT = 2:** Output channels per tile (also referred to as “blocks” in the waveform analysis below).
- **TILE_T = 12:** Output positions per tile. The total output length is $128 - 5 + 1 = 124$ (no padding).

Computed sizes for a single tile:

- Input tile: $4 \text{ in channels} \times 16 \text{ in values} = 64 \text{ words}$
- Weight tile: $2 \text{ out channels} \times 4 \text{ in channels} \times 5 \text{ weights} = 40 \text{ words}$
- Output tile: $2 \text{ out channels} \times 12 \text{ out values} = 24 \text{ words}$
- Total per tile: 128 words (exactly matching the SRAM/Scratchpad capacity)

3.1.4 Hardware Management

The hardware (`conv1d.sv`) processes one tile at a time. The FSM does not iterate over tiles; it is only responsible for:

- Loading the input and weights of the current tile from the SRAM (`FETCH_INPUTS`, `FETCH_WEIGHTS`) to local array: `in_pipe_q` and `wt_pipe_q` (array of 4 elements of 32 bits each).
- Computing the convolution on the tile (`COMPUTE` state).
- Writing the tile output (`WRITE_OUTPUT` state).
- Managing accumulation when `accumulate_i = 1` (`READ_PREV_OUT`, `WAIT_PREV_OUT` states).

Figure 3.3 shows the complete architecture.

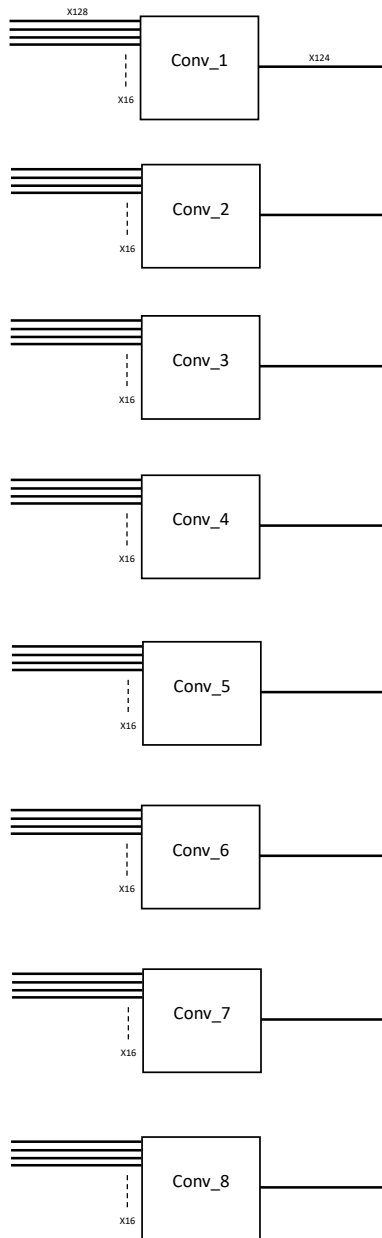


Figure 3.3: Architecture

To illustrate the tiling strategy, Figure 3.4 shows which regions of the input and weight tensors the accelerator processes for the first four tiles.

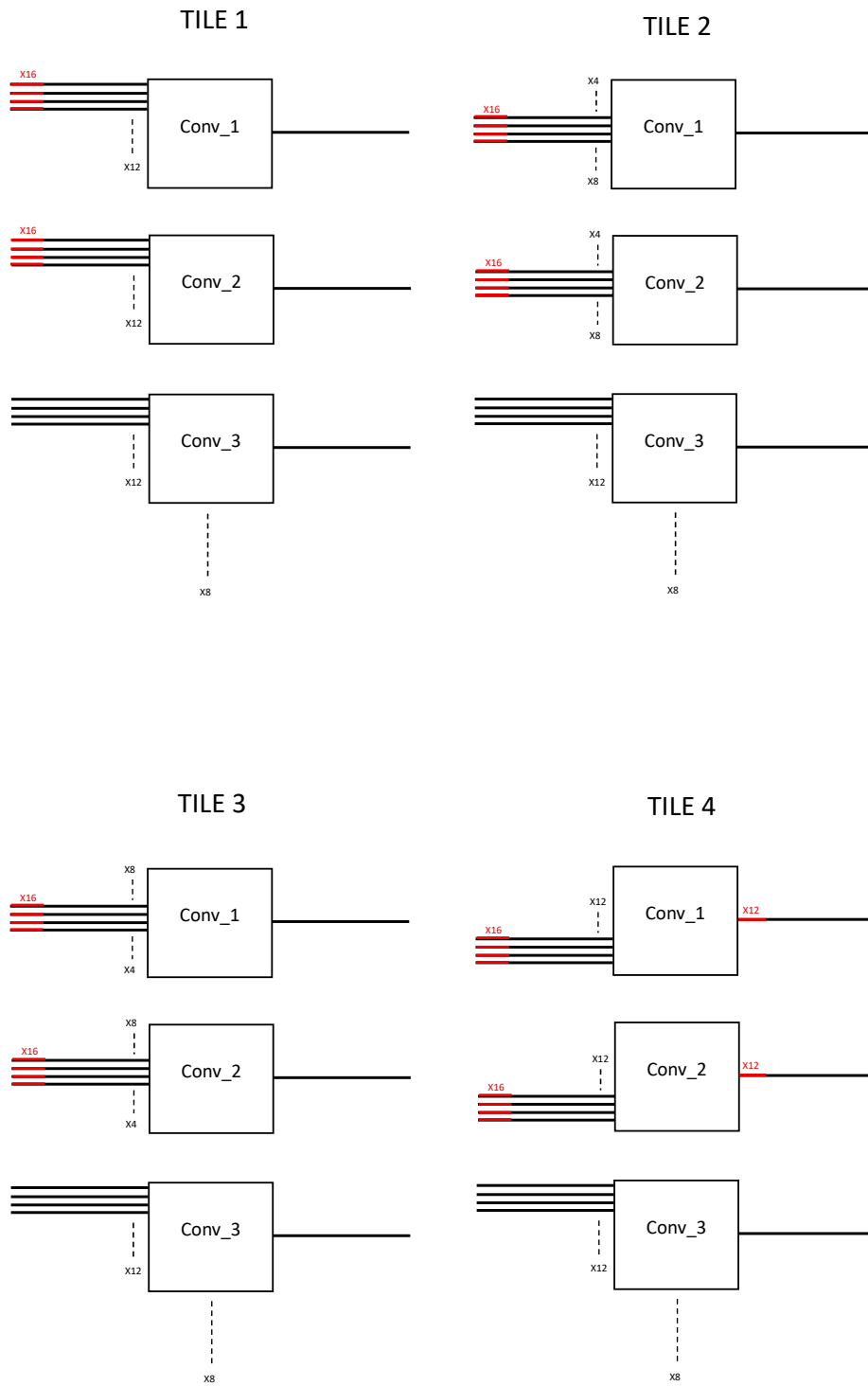


Figure 3.4: Tile 1, 2, 3, 4

Figure 3.5 shows the same for tiles 5 and 6.

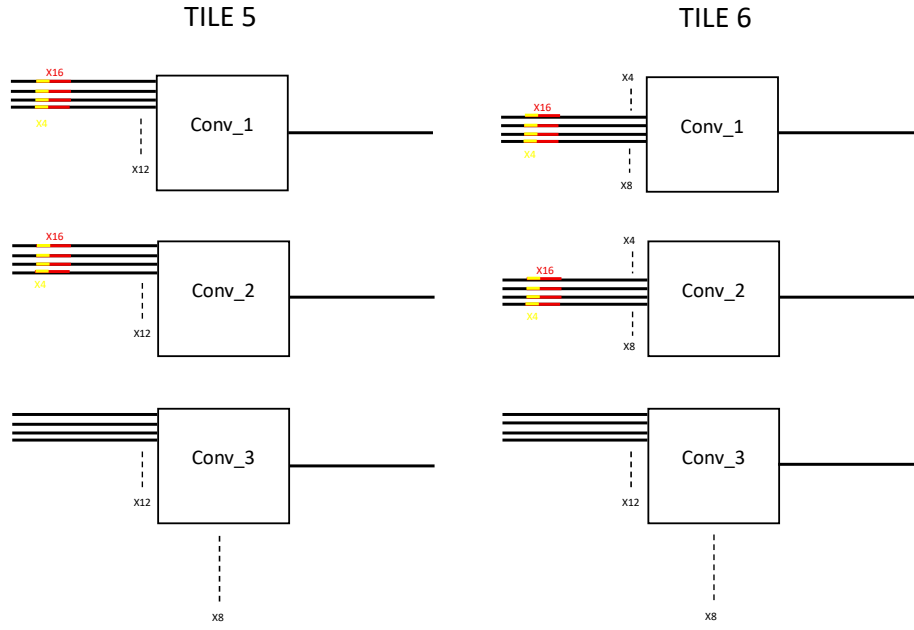


Figure 3.5: Tile 5, 6

In Figures 3.4 and 3.5, the *black* region represents the part of the architecture that remains untouched, meaning that no computations are performed there. The *red* region is used for the computation of the corresponding tile. As shown in the figures, in the first tile, the first 16 input samples of the first 4 input channels of the first 2 blocks are used for the convolution. In the second tile, the first 16 input samples of input channels 4 to 7 are processed, and so on. Only after the fourth tile (when all 4 input-channel tiles have been accumulated) is the final output complete (the first 12 values, not 16, since convolving 16 input values with a kernel of length 5 yields $16 - 5 + 1 = 12$ valid output positions).

Moving to tile 5, input samples from positions 12 to 27 are used. The *yellow* region highlights that 4 input values (positions 12–15) are *overlapped*, meaning that they were already loaded in previous tiles. This overlap is required by the convolution arithmetic (the kernel extends beyond the previous tile boundary). Due to the input window size of 16, only 12 output values are computed for these tiles as well.

Once all output values for the first two blocks have been computed, the program proceeds to compute the outputs for the third and fourth blocks, and so on.

3.1.5 Communication

The design ensures safe shared access to the single-port SRAM, giving priority to the accelerator during computation and allowing the host to load inputs/weights and read outputs when the accelerator is in the IDLE or DONE state. The main components are:

- `conv1d.sv`: top-level accelerator module, contains FSM, multiplexers, and OBI-SRAM bridge.
- `conv1d_obi.sv`: OBI wrapper that instantiates `conv1d` and control registers.
- `obi_sram_shim.sv`: OBI-to-SRAM bridge with simplified signals.
- `conv1d_sram_wrapper.sv`: SRAM wrapper (uses `tc_sram` for a simulatable model).
- `conv1d_sram_pkg.sv`: defines `sram_req_t` and `sram_rsp_t`.

3.1.6 OBI Protocol (External Bus)

OBI is the protocol used by the host to access the SRAM.

- Request: `sbr_obi_req_t` with `req` (request flag), `we` (write enable), `addr[31:0]`, `wdata[31:0]`, `be[3:0]` (byte enable).
- Response: `sbr_obi_rsp_t` with `gnt`, `rvalid`, `r.rdata`.

The `obi_sram_shim.sv` bridge converts the OBI protocol into SRAM signals:

```

1 assign req_o    = obi_req_i.req;
2 assign we_o     = obi_req_i.a.we;
3 assign addr_o   = obi_req_i.a.addr;
4 assign wdata_o  = obi_req_i.a.wdata;
5 assign be_o     = obi_req_i.a.be;
6
7 always_comb begin
8     obi_rsp_o    = '0;
9     obi_rsp_o.gnt = gnt_i;
10    obi_rsp_o.rvalid = rvalid_q;
11    obi_rsp_o.r.rdata = rdata_i;
12    obi_rsp_o.r.rid  = id_q;
13    obi_rsp_o.r.err  = 1'b0;
14 end

```

3.1.7 Arbitration and Multiplexer

SRAM access is arbitrated in the `conv1d.sv` module, by using a multiplexer. This mux is driven by the signal `ext_mem_gnt`. If the FSM is in IDLE or DONE state, the SRAM can communicate only with the cpu (`ext_mem_gnt = 1`), otherwise with the accelerator (`ext_mem_gnt = 0`).

- When `ext_mem_gnt = 1` priority is given to the external OBI.
- When `ext_mem_gnt = 0` priority is given to the internal FSM.

Multiplexer:

```

1 // Memory
2 // CPU can access only when accelerator is in IDLE or DONE state
3 assign ext_mem_gnt = (state_q == IDLE) || (state_q == DONE);
4
5 // Memory request multiplexer
6 // If accelerator is in IDLE or DONE state, CPU can communicate with SRAM
7 always_comb begin : mem_req_mux
8     mem_req = ext_mem_gnt ? ext_mem_req : int_mem_req;
9 end

```

Since the OBI address is byte-addressable, it needs to be converted to a word-addressable format, because the scratchpad stores data as 32-bit words, not as 8-bit bytes. To perform the conversion from byte addressing to word addressing, the two least significant bits are discarded, since they are used to select the byte within a word. To keep the address width unchanged, the most significant bit range is extended by two bits.

Final address to the SRAM:

```
1 addr_i = ext_mem_gnt ?
2     ext_mem_req.addr[AddrWidth-1+2:2] :
3     int_mem_req.addr[AddrWidth-1:0];
```

3.2 Architecture verification

The architecture has been verified via simulation. Figs. 3.6, 3.7, 3.8, 3.9 show the obtained waveforms. In particular, Fig. 3.6 shows that the pipeline is not sensitive to the memory latency and that the protocol works as expected. The total simulation time to execute the full conv1d computation (128 input length, 16 input channels, 5 kernel length, 8 output channels) is 576738 ps; with the Verilator default clock period of 2 ps, this corresponds to 288369 clock cycles.

3.2.1 Testbench

Our testbench is composed of 10 steps, in which each step is represented by a case statement. These statements are controlled using the variable `step_cnt`.

Step 0 In this step, the address is calculated in order to fetch and place the correct input values for the tile that is going to be computed, moving the data from the generated data arrays (from `data.h`) to the scratchpad (SRAM), while also performing the conversion from 8-bit to 32-bit with sign extension. According to our tiling strategy, in this step 16 values for each of the 4 input channels are loaded into the scratchpad, for a total of 64 32-bit words.

Step 1 It is the same as **step 0**, but it manages the weights/kernels. The conversion from 8-bit to 32-bit with sign extension is always performed. According to our tiling strategy, in this step 5 values for each of the 4 input channels for 2 blocks are loaded into the scratchpad, for a total of 40 32-bit words.

Step 2 to 4 In these steps the testbench is waiting for all word loading

Step 5 Start the accelerator. The `CONTROL` register is written with 1 for the `START` bit and with 1 or 0 (depending on whether it is the first tile of the block) for `ACCUMULATE` bit.

Step 6 The testbench waits for the termination of the accelerator computation by waiting for the completion interrupt (`irq_received`), triggered by the accelerator's `done_o` output. When the accelerator terminates, the `START` bit is cleared (In this way the FSM can go to IDLE state).

Step 7 In this step, if the computation of the tile is not concluded, this step is skipped. Otherwise, if the computation of the tile is concluded, the output values computed by the accelerator and stored in the scratchpad (SRAM) are loaded into an array of 124×8 elements (992 total), each 32 bits wide, (called `computed_outputs`) in the testbench. In this way, when the accelerator finishes processing all the tiles, the testbench can compare the results with the golden ones by looking at this array.

Step 8 This step is used to determine whether the program has processed all tiles, all inputs, and all output values, that is, whether it has finished the entire execution; otherwise, the steps are repeated starting from step 0.

Step 9 In this step, the comparison between the values computed by the accelerator and the golden ones are performed by simply checking whether the two arrays are the same. A notification of the result is printed in the terminal.

3.2.2 Simulation results

The commands to make the architecture work and to perform the simulations are:

```

1 make gen-registers
2 make gen-data
3 make gen-data APP_PARAMS="--in_len 128 --in_ch 16 --k_len 5 --k_num 8" # For a
  complete execution
4 make gen-data APP_PARAMS="--in_len 16 --in_ch 4 --k_len 5 --k_num 2" # For an
  execution of one tile
5 make verilator-sim
6 make waves

```

Listing 3.4: Commands

For the execution of one tile, the result on the terminal is:

```

1 Running Verilator simulation...
2 [LOG ] conv1d_tb.cpp:202 > [ 0] Starting simulation...
3 [LOG ] conv1d_tb.cpp:421 > [ 3316] Verification complete. Total errors: 0 / 24
  outputs
4 [LOG ] conv1d_tb.cpp:469 > [ 3338] End of simulation reached: terminating.
5 [OK! ] conv1d_tb.cpp:497 > [ 3338] CHECKS PASSED > errors: 0 (checked 0
  transactions)
6 Simulation passed successfully

```

For the execution of the entire program:

```

1 Running Verilator simulation...
2 [LOG ] conv1d_tb.cpp:202 > [ 0] Starting simulation...
3 [LOG ] conv1d_tb.cpp:421 > [576716] Verification complete. Total errors: 0 /
  992 outputs
4 [LOG ] conv1d_tb.cpp:469 > [576738] End of simulation reached: terminating.
5 [OK! ] conv1d_tb.cpp:497 > [576738] CHECKS PASSED > errors: 0 (checked 0
  transactions)
6 Simulation passed successfully

```

In both simulations, all the results are correct. In the one-tile simulation, it checks only 24 values; in the full simulation, it checks all $124 \times 8 = 992$ values.

3.2.3 Timing analysis and waveforms

The simulation lasts for 576738 ps (288369 clock cycles at a 2 ps clock period), and the computation time for a single tile is approximately 1638 cycles. Therefore, a total of (288369/1638) tiles are computed. This result is correct because, with 4 tiles, 12 output values are produced for 2 blocks. Consequently, to compute all 124 output values, 11 groups of 4 tiles are required, corresponding to 44 tiles. Since there are 8 blocks computed two at a time, the program requires a total of $44 \times 4 = 176$ tiles.

Figure 3.6 shows the timing diagram corresponding to the start of the program. Since this is the initial phase, the **ACCUMULATE** bit is set to 0, as no partial sums have been computed yet. Consequently, the FSM transitions from the **IDLE** state to **FETCH_INPUTS**, and in the **in_pipe_q** array it is possible to observe the values being loaded from the scratchpad. The same applies to the following state, **FETCH_WEIGHTS**, where the values are loaded into the **wt_pipe_q** array.

A wait cycle is then required to account for the SRAM latency, after which the **COMPUTE** state is entered. Since the multipliers and the adder tree are connected in a combinational manner, each time the input values change, the result of the computation (stored in **add_tree_out**) changes accordingly. However, the correct value to be considered is produced only during the **COMPUTE** state, because in this state the accumulation term **accum_q** is taken into account.

The program then continues by iterating through the same sequence of states.

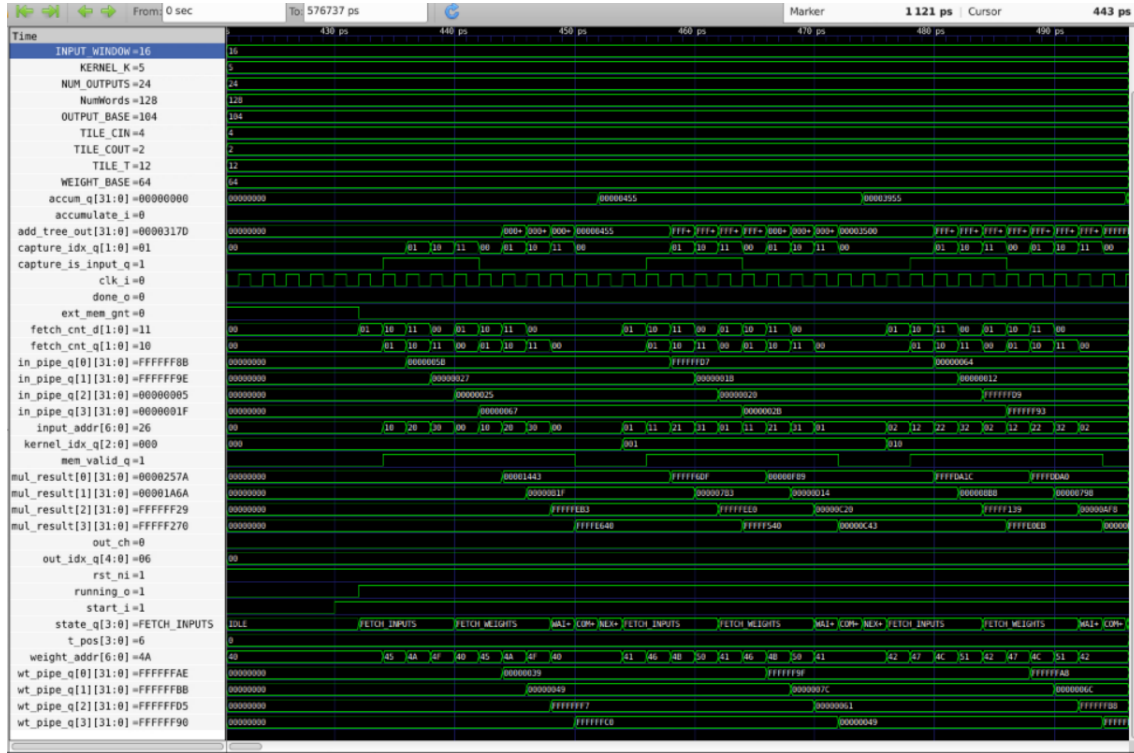
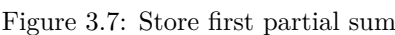


Figure 3.6: Program start

Figure 3.7 shows the **WRITE_OUTPUT** state. This state is entered when five kernel values have been loaded and computed, because at that point a partial sum is produced for the first output value. After this write operation, the counter **out_idx_q** is incremented by one, indicating that the program will now compute the partial sum for the next output value.



Moreover, since a tile computes 12 output values for 2 different blocks, it can be seen that in the middle of the tile the `out_ch` signal changes from 0 to 1, indicating a switch of the output channel from block 1 to block 2. Additionally, the counter `out_idx_q`, which represents the output position, changes from 0 to 23 (24 values corresponding to 2 outputs of 12 values each).

Moreover, at the beginning of the second tile, the `accumulate_i` signal is set to 1, because after the first tile partial sums must be taken into account.

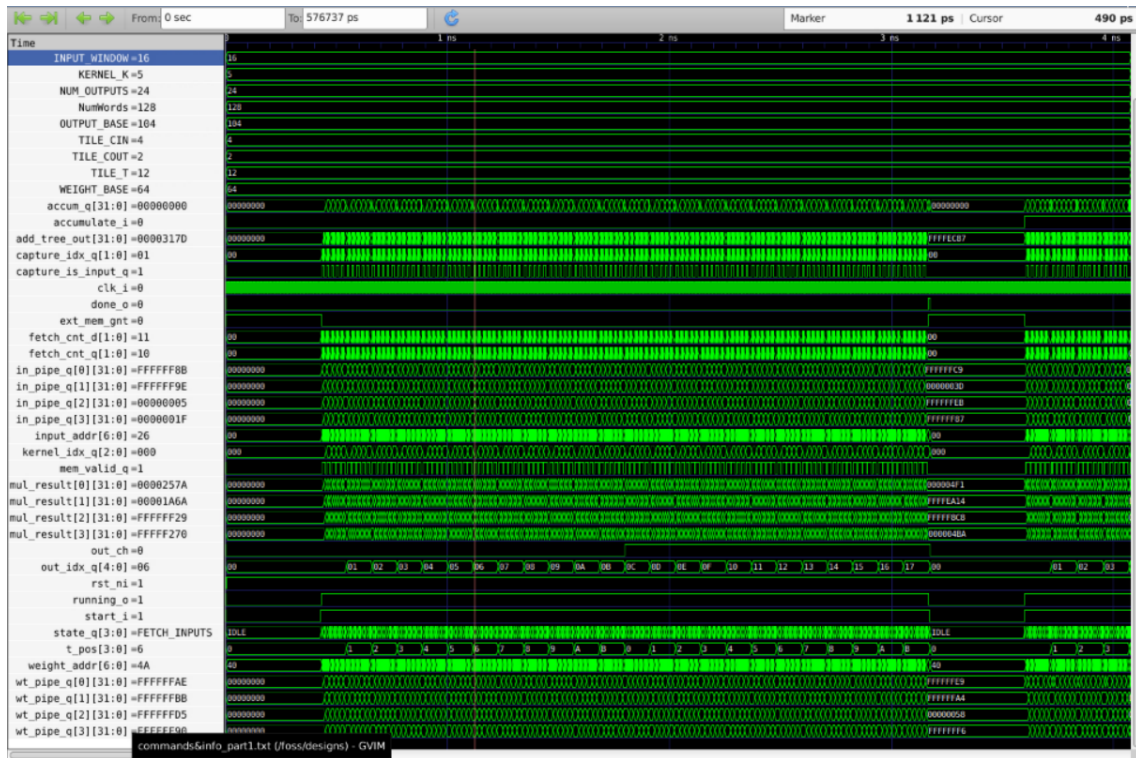


Figure 3.8: First tile processed

Figure 3.9 shows the first 4 processed tiles.

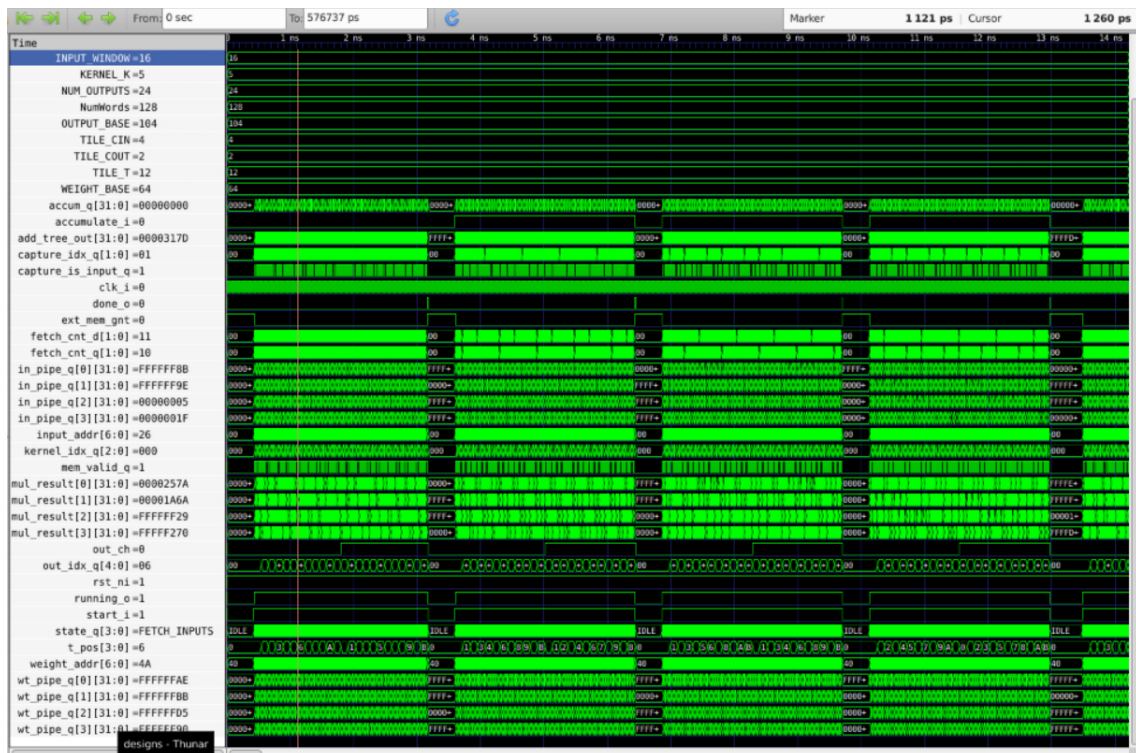


Figure 3.9: First 4 tiles processed

3.3 Including the SSRAM in the architecture

In Part 2, the conv1d accelerator developed in Part 1 is integrated into the Croc SoC which is organized into two domains:

- **croc_domain:** contains the CVE2 (Ibex) RISC-V core, an OBI crossbar interconnect, 16 SRAM banks (1024×64 each), a timer, debug module, UART, and GPIO peripherals.
- **user_domain:** hosts custom accelerators connected to the OBI crossbar. This is where the conv1d accelerator is instantiated.

3.3.1 Accelerator integration

The accelerator was integrated using the Bender dependency manager. In the top-level `Bender.yml`, the conv1d accelerator was added as a vendor dependency:

```

1 vendor_package:
2   - name: conv1d_accel
3     target_dir: rtl/user_domain/conv1d
4     upstream: { git: "https://github.com/.../conv-1d-group-04", rev: "a442bb7"
5       ..." }
6     mapping:
7       - { from: 'hw/', to: 'hw/' }
8       - { from: 'sw/', to: '../.../sw/control_reg/conv1d/' }
9       - { from: 'Bender.yml', to: 'Bender.yml' }
```

Listing 3.5: Bender.yml vendor entry for conv1d

Running `make bender-update` vendors the RTL source files into `rtl/user_domain/conv1d/`. The accelerator is then instantiated in `user_domain.sv`:

```

1 // Conv1d Subordinate
2 conv1d_obi u_conv1d_obi (
3   .clk_i      ( clk_i ),
4   .rst_ni     ( rst_ni ),
5   .obi_req_i  ( user_conv1d_obi_req ),
6   .obi_rsp_o  ( user_conv1d_obi_rsp ),
7   .reg_req_i  ( conv1d_reg_req ),
8   .reg_rsp_o  ( conv1d_reg_rsp ),
9   .done_int_o ( interrupts_o[1] )
10 );
```

Listing 3.6: Conv1d instantiation in user_domain.sv

The accelerator is mapped into the RISC-V address space as shown in Table 3.1.

Peripheral	Base Address	Size
Conv1D Scratchpad	0x2001_0000	512B
Conv1D CSR	0x2002_1000	4KB

These addresses are defined in `config.h` and used by the firmware to access the accelerator:

```

1 #define USER_BASE_ADDR      0x20000000
2
3 #define USER_CONV1D_START_ADDRESS (USER_BASE_ADDR + 0x00010000) // 0
4   x20010000
5
6 #define USER_CONV1D_SIZE      0x00010000
7
8 #define USER_CONV1D_CSR_START_ADDRESS (USER_BASE_ADDR + 0x00021000) // 0
9   x20021000
10
11 #define USER_CONV1D_CSR_SIZE  0x00001000
```

Listing 3.7: Accelerator address definitions (config.h)

The scratchpad is accessed as a `volatile uint32_t * array` at `USER_CONV1D_START_ADDRESS`, while the control/status registers are accessed at `USER_CONV1D_CSR_START_ADDRESS` with the auto-generated offsets from the HJSON specification.

Note that the 4 KB CSR region in Table 3.1 is the minimum page-aligned address range required by the OBI bus decoder for routing, not physical storage: only two 32-bit registers (8 bytes at offsets 0x00 and 0x04) are actually implemented. Accesses to other addresses within the region return a bus error. Similarly, the scratchpad's bus allocation in `user_pkg.sv` is 64 KB (0x0001_0000), though only 512 bytes (128 words) are physically instantiated.

3.3.2 SRAM implementation for synthesis

The accelerator's internal scratchpad memory (128 words $\times$ 32 bits) requires a physical SRAM macro for synthesis. The IHP SG13G2 130nm PDK does not provide a native 128 $\times$ 32 macro, so the `tc_sram_impl` wrapper maps the logical 128 $\times$ 32 memory onto a physical `RM_IHPG13_1P_64x64_c2_bm_bist` macro (64 words $\times$ 64 bits) using bit interleaving.

The bit interleaving technique maps each 32-bit data word into alternating even/odd bits of the 64-bit SRAM data bus. The address LSB selects which set of interleaved bits is active:

```

1  // 128x32 uses 64x64 macro with bit interleaving
2  always_comb begin : gen_bit_interleaving
3      for (int i = 0; i < 32; i++) begin
4          wdata64[2*i]   = wdata_i[0][i]; // even bits (addr LSB=0)
5          bm64[2*i]     = bm[0][i] & ~addr_i[0][0];
6          wdata64[2*i+1] = wdata_i[0][i]; // odd bits (addr LSB=1)
7          bm64[2*i+1]   = bm[0][i] & addr_i[0][0];
8
9          if(~sel_q) rdata_o[0][i] = rdata64[2*i]; // even bits
10         else      rdata_o[0][i] = rdata64[2*i+1]; // odd bits
11     end
12 end

```

Listing 3.8: Bit interleaving logic in `tc_sram_impl.sv` (128x32 case)

This approach halves the number of SRAM rows required (128 $\rightarrow$ 64) while doubling the data width (32 $\rightarrow$ 64 bits). The bit mask (`bm64`) ensures that only the correct half of the 64-bit word is written on each access. A registered select signal (`sel_q`) captures the address LSB during read requests to correctly demultiplex the output one cycle later (matching the SRAM read latency).

For simulation, the behavioral model `conv1d_sram_wrapper` (which uses `tc_sram`) is used instead, by toggling comments in two files. In `conv1d.sv`, the synthesis block is commented out and the simulation block is uncommented:

```

1  // SIMULATION SRAM -- uncomment for Verilator HW vs SW testing
2  conv1d_sram_wrapper #(
3      .NUM_WORDS (NumWords),
4      .DATA_WIDTH(32'd32)
5  ) u_internal_mem ( ... );
6
7  // SYNTHESIS SRAM -- comment out for simulation
8  /* tc_sram_impl #(
9      .NumWords (NumWords),
10     .DataWidth (32),
11     .NumPorts (1),
12     .Latency (1)
13 ) u_internal_mem ( ... ); */

```

Listing 3.9: SRAM toggle in `conv1d.sv`

Correspondingly, in the accelerator's `Bender.yml`, the simulation SRAM sources must be uncommented:

```

1  # Simulation SRAM -- uncomment for Verilator HW vs SW testing
2  - hw/simulation/conv1d_sram_wrapper.sv
3  - hw/vendor/pulp-platform-tech-cells-generic/src/rtl/tc_sram.sv

```

Listing 3.10: SRAM toggle in `Bender.yml`

Before synthesis, these changes must be reversed: the `tc_sram_impl` block is uncommented (so that the IHP SG13G2 SRAM macro is used), and the simulation sources are commented out in `Bender.yml`.

3.4 Complete architecture verification

3.4.1 Driver implementation

A driver was developed to control the accelerator from the RISC-V core via memory-mapped I/O. The driver provides four functions:

```

1  #include "config.h"                // USER_CONV1D_CSR_START_ADDRESS
2  #include "conv1d_control_reg.h"    // Auto-generated register offsets/bits
3
4  void conv_accel_init() {
5      *(volatile uint32_t *) (USER_CONV1D_CSR_START_ADDRESS
6          + CONV1D_CONTROL_CONTROL_REG_OFFSET) = 0;
7  }
8
9  void conv_accel_start() {
10     *(volatile uint32_t *) (USER_CONV1D_CSR_START_ADDRESS
11         + CONV1D_CONTROL_CONTROL_REG_OFFSET) |=
12         (1 << CONV1D_CONTROL_CONTROL_START_BIT);
13 }
14
15 void conv_accel_wait() {
16     while (!(*(volatile uint32_t *) (USER_CONV1D_CSR_START_ADDRESS
17         + CONV1D_CONTROL_STATUS_REG_OFFSET)
18         & (1 << CONV1D_CONTROL_STATUS_DONE_BIT))) { /* Busy wait */ }
19     *(volatile uint32_t *) (USER_CONV1D_CSR_START_ADDRESS
20         + CONV1D_CONTROL_CONTROL_REG_OFFSET) &=
21         ~(1 << CONV1D_CONTROL_CONTROL_START_BIT);
22 }
23
24 void conv_accel_set_accumulate(uint8_t accumulate) {
25     if (accumulate)
26         *(volatile uint32_t *) (USER_CONV1D_CSR_START_ADDRESS
27             + CONV1D_CONTROL_CONTROL_REG_OFFSET) |=
28             (1 << CONV1D_CONTROL_CONTROL_ACCUMULATE_BIT);
29     else
30         *(volatile uint32_t *) (USER_CONV1D_CSR_START_ADDRESS
31             + CONV1D_CONTROL_CONTROL_REG_OFFSET) &=
32             ~(1 << CONV1D_CONTROL_CONTROL_ACCUMULATE_BIT);
33 }

```

Listing 3.11: Hardware accelerator driver (conv_accel.c)

The macro names (e.g., CONV1D_CONTROL_CONTROL_START_BIT, CONV1D_CONTROL_STATUS_DONE_BIT) are auto-generated from the HJSON register specification as done in Part 1. The driver communicates with the accelerator through the CSR register block at USER_CONV1D_CSR_START_ADDRESS.

Completion detection uses polling (busy-wait on the DONE bit) rather than the available interrupt line (done_int_o is connected to interrupts_o[1] in user_domain.sv). Polling was chosen because the firmware has no other work to perform while the accelerator runs, so the simplicity of a polling loop outweighs the overhead of configuring an interrupt handler, vector table, and context save/restore on the CVE2 core.

3.4.2 Hardware-accelerated convolution firmware

In Part 1, the tiling loops were managed by the C++ testbench driving the OBI bus directly. In Part 2, the same tiling logic is executed as bare metal firmware on the CVE2 RISC-V core, accessing the scratchpad through memory-mapped writes.

The firmware function conv1d_hw() implements software tiling to process convolutions larger than the accelerator's 128-word scratchpad. The tiling is organized as a 3-level nested loop with the order: output channels (outermost) → temporal positions → input channels (innermost). This nesting order is chosen so that all input channel tiles are processed consecutively, accumulating partial sums in the scratchpad via the ACCUMULATE flag. Once the innermost loop completes, the full output for that pair is ready and can be read back in a single pass. Any other loop order would require either storing intermediate partial sums in main memory or re-loading outputs for accumulation, both of which would increase data movement.

```

1  #define TILE_CIN    4    // Input channels per tile (matches 4 HW multipliers)
2  #define TILE_COUT   2    // Output channels per tile
3  #define TILE_T      12   // Output time samples per tile
4  #define HW_K        5    // Kernel length (fixed in hardware)
5  #define INPUT_WINDOW (TILE_T + HW_K - 1) // 16 input samples needed
6
7  #define SP_INPUT_BASE 0    // Scratchpad word addresses
8  #define SP_WEIGHT_BASE 64
9  #define SP_OUTPUT_BASE 104

```

Listing 3.12: Hardware tiling constants and scratchpad layout (conv1d.c)

The `conv1d_hw()` function implements the 3-level tiling loop. A pointer to the scratchpad is obtained via the memory-mapped address: `volatile uint32_t *scratchpad = (volatile uint32_t *)USER_CONV1D_START_ADDRESS`. For each tile, the CPU first copies input samples and filter weights into the scratchpad, then triggers the accelerator:

```

1  // === Copy inputs to scratchpad ===
2  // Layout: inputs[cin][t] at word address cin * INPUT_WINDOW + t
3  for (uint32_t c = 0; c < actual_cin; c++) {
4      uint32_t global_c = n_start + c;
5      for (uint32_t t = 0; t < INPUT_WINDOW; t++) {
6          uint32_t global_t = t_start + t;
7          int8_t input_val = 0;
8          if (global_t < input_len)
9              input_val = mem2d(in_data, input_len, global_c, global_t);
10         scratchpad[SP_INPUT_BASE + c * INPUT_WINDOW + t] = (int32_t)input_val;
11     }
12 }
13 // Zero-pad remaining input channels if actual_cin < TILE_CIN
14 for (uint32_t c = actual_cin; c < TILE_CIN; c++)
15     for (uint32_t t = 0; t < INPUT_WINDOW; t++)
16         scratchpad[SP_INPUT_BASE + c * INPUT_WINDOW + t] = 0;
17
18 // === Copy weights to scratchpad ===
19 // Layout: weights[cout][cin][k] at word address
20 //      cout * (TILE_CIN * HW_K) + cin * HW_K + k
21 for (uint32_t m = 0; m < (m_end - m_start); m++) {
22     uint32_t global_m = m_start + m;
23     for (uint32_t c = 0; c < actual_cin; c++) {
24         uint32_t global_c = n_start + c;
25         for (uint32_t k = 0; k < HW_K; k++) {
26             int8_t weight_val = mem3d(filter, input_ch,
27                                     KERNEL_LEN, global_m, global_c, k);
28             scratchpad[SP_WEIGHT_BASE + m*(TILE_CIN*HW_K) + c*HW_K + k]
29                 = (int32_t)weight_val;
30         }
31     }
32     // Zero-pad remaining cin channels
33     for (uint32_t c = actual_cin; c < TILE_CIN; c++)
34         for (uint32_t k = 0; k < HW_K; k++)
35             scratchpad[SP_WEIGHT_BASE + m*(TILE_CIN*HW_K) + c*HW_K + k] = 0;
36 }
37
38 // === Trigger accelerator ===
39 conv_accel_set_accumulate(cin_tile > 0 ? 1 : 0);
40 conv_accel_start();
41 conv_accel_wait();

```

Listing 3.13: Input and weight copy to scratchpad (conv1d.c, inner loop body)

The `mem2d` and `mem3d` macros index into the flat 8-bit input and filter arrays. Each 8-bit value is sign-extended to 32 bits before being written to the scratchpad, matching the hardware's 32-bit word width. Zero-padding ensures that unused input channels and weights are cleared when the tile dimensions do not evenly divide the problem size.

The `ACCUMULATE` flag is set to 0 for the first input channel tile and to 1 for subsequent tiles,

allowing partial sums to be accumulated across input channel tiles. After all input channel tiles are processed, the CPU reads the output values from the scratchpad:

```

1 // == Read outputs from scratchpad ==
2 for (uint32_t m = 0; m < (m_end - m_start); m++) {
3     uint32_t global_m = m_start + m;
4     for (uint32_t t = 0; t < actual_t_samples; t++) {
5         uint32_t global_t = t_start + t;
6         output_data[global_m * output_len + global_t] =
7             (int32_t)scratchpad[SP_OUTPUT_BASE + m * TILE_T + t];
8     }
9 }

```

Listing 3.14: Output read-back from scratchpad (conv1d.c)

3.4.3 Test application

The test application (main.c) performs both hardware-accelerated and software convolution, timing each with the RISC-V mcycle performance counter:

```

1 int main() {
2     uart_init();
3     conv_accel_init();
4
5     // Hardware-accelerated convolution (timed)
6     t_start = get_mcycle();
7     conv1d_hw(A, F, R_hw, INPUT_LEN, INPUT_CH, K_NUM);
8     t_end = get_mcycle();
9     printf("HW cycles: %u\n", t_end - t_start);
10
11    // Software convolution (timed)
12    t_start = get_mcycle();
13    conv1d_sw(A, F, R_sw, INPUT_LEN, INPUT_CH, K_NUM);
14    t_end = get_mcycle();
15    printf("CPU cycles: %u\n", t_end - t_start);
16
17    // Verify both against golden reference
18    if (memcmp(R_sw, R, R_SIZE) != 0) {
19        printf("CPU result mismatch!\n");
20        return -1;
21    }
22    if (memcmp(R_hw, R, R_SIZE) != 0) {
23        printf("HW result mismatch!\n");
24        return -1;
25    }
26    printf("Success!\n");
27    return 0;
28 }

```

Listing 3.15: Test application structure (main.c)

3.4.4 Software reference implementation

The software convolution function `conv1d_sw()` serves as the golden reference executed on the CVE2 core. It implements a naive convolution with four nested loops:

```

1 void conv1d_sw(const int8_t *input_data, const int8_t *filter,
2               int32_t *output_data, const uint32_t input_len,
3               const uint32_t input_ch, const uint32_t output_ch) {
4     uint32_t padded_input_len = input_len + 2 * PAD;
5     uint32_t output_len = ((padded_input_len - KERNEL_LEN) / STRIDE) + 1;
6
7     for (uint32_t m = 0; m < output_ch; m++) {           // output channels
8         for (uint32_t o = 0; o < output_len; o++) {      // output positions
9             int32_t accumulated_output = 0;
10            for (uint32_t n = 0; n < input_ch; n++) {     // input channels

```

```

11         int32_t input_start = o * STRIDE - PAD;
12         int32_t sum = 0;
13         for (uint32_t k_idx = 0; k_idx < KERNEL_LEN; k_idx++) {
14             int32_t input_index = input_start + k_idx;
15             int8_t input_value;
16             if (input_index < 0 || input_index >= (int32_t)input_len)
17                 input_value = 0; // zero-padding
18             else
19                 input_value = mem2d(input_data, input_len,
20                                     n, input_index);
21             int8_t filter_value = mem3d(filter, input_ch,
22                                         KERNEL_LEN, m, n, k_idx);
23             sum += input_value * filter_value;
24         }
25         accumulated_output += sum;
26     }
27     output_data[m * output_len + o] = accumulated_output;
28 }
29 }
30 }

```

Listing 3.16: Software convolution (conv1d_sw in conv1d.c)

The `mem2d` and `mem3d` macros provide row-major indexing into the flat input and filter arrays (e.g., `mem2d(ptr, cols, row, col)` expands to `ptr[row * cols + col]`). Padding is handled by checking whether the input index falls outside the valid range `[0, input_len)`, substituting zero when it does. The function accumulates partial sums across all input channels for each output position, producing 32-bit output values from 8-bit inputs and weights.

In the test configuration, `PAD=0` and `STRIDE=1` (set by `datagen.py` in `data.h`), so the padding branches are never taken and the output length reduces to $128 - 5 + 1 = 124$, matching the hardware's valid-only convolution. The `conv1d_sw` function supports arbitrary padding and stride for generality, but only the valid convolution case is exercised here.

3.4.5 Simulation results

The SoC simulation was run with Verilator using the following commands:

```

1 make gen-data APP_PARAMS="--in_len 128 --in_ch 16 --k_len 5 --k_num 8"
2 make sw PROG=conv1d
3 make verilator

```

Listing 3.17: Part 2 simulation commands

The full simulation output is:

```

1 @      41100ns | [JTAG] Initialization success
2 @      41100ns | [JTAG] Writing 0x12345678 to 0x10000000
3 @      93650ns | [JTAG] Read 0x12345678 from 0x10000000
4 @      93650ns | [JTAG] Read back correct data
5 @      93650ns | [JTAG] Loading binary from ../sw/bin/main.hex
6 @     42597650ns | [CORE] Start fetching instructions
7 @     42615600ns | [CORE] Wait for end of code...
8 @     69915900ns | [UART] HW cycles: 514400
9 @     897363900ns | [UART] CPU cycles: 16515351
10 @     899083900ns | [UART] Success!

```

The output shows the complete SoC boot sequence. First, the JTAG debug module initializes and performs a read-back test on the main SRAM at `0x1000_0000` to verify bus connectivity. It then loads the firmware binary (`main.hex`) into memory, which takes approximately 42ms of simulation time due to the slow JTAG serial interface. Once loading completes, the CVE2 core starts fetching instructions. The three UART lines at the end are printed by the firmware: the hardware-accelerated convolution result (514,400 mcycle cycles), the software convolution result (16,515,351 cycles), and the final verification status.

Both the hardware and software convolution results match the golden reference data (generated by `datagen.py`), confirming correct accelerator integration into the Croc SoC.

3.4.6 Performance comparison

Table 3.2 summarizes the performance comparison between the hardware accelerator and the software implementation for the full convolution (128 input length, 16 input channels, kernel length 5, 8 output channels).

Table 3.2: Hardware vs software performance comparison

Implementation	Cycles	Speedup
Hardware (Accelerator)	514,400	—
Software (CPU only)	16,515,351	—
Speedup	—	32.1×

HW cycle breakdown

The hardware cycle count of 514,400 is measured using the RISC-V `mcycle` performance counter around the `conv1d_hw()` function call. This measures total clock cycles including CPU time for scratchpad data transfer, busy-wait polling during accelerator execution, and output read-back.

A total of 176 tiles are computed ($4 \text{ cout_tiles} \times 11 \text{ t_tiles} \times 4 \text{ cin_tiles}$), giving:

$$514,400 / 176 = 2,923 \text{ cycles/tile}$$

For comparison, the Part 1 standalone simulation completed the same convolution in 288,369 clock cycles (576,738 ps at a 2 ps clock period), measured by the testbench simulation-time counter. This gives $288,369/176 = 1,638$ cycles/tile. The Part 1 count is lower because the testbench drives OBI transactions directly with minimal protocol overhead, whereas the Part 2 `mcycle` counter includes CPU instruction execution overhead for copying data between main SRAM and the scratchpad, tiling loop control, and busy-wait polling.

The actual wall-clock simulation time in Part 2 is much longer: the `HW_cycles` UART message appears at approximately 70 Mns (at 20 MHz = 50 ns/cycle), corresponding to roughly 546,000 clock cycles from the start of instruction fetch (at 42.6 Mns). The `mcycle` counter shows 514,400 because it only measures the interval around the `conv1d_hw()` call, not the full simulation.

SW cycle breakdown

The software implementation requires 16,515,351 CPU cycles. The total number of scalar multiply-accumulate (MAC) operations is:

$$8 \times 16 \times 124 \times 5 = 79,360 \text{ MACs}$$

This gives $16,515,351 / 79,360 = 208.1$ cycles/MAC. This figure includes input and weight loads, multiplication, accumulation, address calculation, and loop-control overhead on CVE2.

The hardware accelerator achieves a **32.1×** **speedup** ($16,515,351/514,400$) over the pure software implementation. The advantage comes from: (1) 4 parallel multipliers computing 4 MACs in a single cycle, (2) dedicated FSM control with no instruction fetch/decode overhead, and (3) direct single-cycle SRAM access without load/store instructions.

3.5 Complete architecture: logic synthesis results

The Croc SoC (including the conv1d accelerator) was synthesized using Yosys, targeting the IHP SG13G2 130nm standard cell library. The synthesis commands are:

```
1 make yosys-flist
2 make yosys
```

Listing 3.18: Synthesis commands

Synthesis completed cleanly with “Found and reported 0 problems”.

3.5.1 Synthesis area results

Table 3.3 shows the area breakdown by major module from the Yosys hierarchy report.

Table 3.3: Synthesis area breakdown by module (Yosys)

Module	Area (μm^2)
croc_chip (total logic)	785,736
croc_domain	181,387
user_domain	191,562
CVE2 core (core_wrap)	69,980

Table 3.4: Synthesis summary metrics (Yosys)

Metric	Value
Total Logic Area	785,736 μm^2
Total Cells	56,849
Flip-Flops	5,034
SRAM Macros (1024×64)	16
SRAM Macros (64×64)	1 (conv1d)

3.5.2 Explanations and comments

The user_domain (191,562 μm^2) is slightly larger than the croc_domain (181,387 μm^2) in terms of area. This is because the conv1d accelerator includes 4 hardware multipliers, an adder tree, pipeline registers, and the OBI wrapper with its register interface, which together contribute significant combinational and sequential logic.

The total chip area is dominated by the SRAM macros. The 16 main SRAM banks and the conv1d scratchpad SRAM macro occupy the majority of the silicon area. The logic area (785,736 μm^2) represents only about 13% of the total active area reported after place and route (5.87 mm²), with the remaining area consumed by SRAM macros and I/O pads.

3.6 Complete architecture: place and route

Place and route was performed using OpenROAD with the following command:

```
1 make openroad
```

Listing 3.19: Place and route command

3.6.1 Floorplan

The die area is $5100 \times 5100 \mu\text{m}$ (26.01 mm^2). The 16 main SRAM banks (1024×64) are arranged in two columns of 8 on the left and right sides. The conv1d accelerator's 64×64 SRAM macro is placed at the bottom center of the core area:

```
1 # Place conv1d accelerator SRAM (64x64) at bottom center
2 set conv1d_X [expr $floor_midpointX - $RamSize64x64_W/2]
3 set conv1d_Y [expr $floor_bottomY + 50]
4 placeInstance $CONV1D_SRAM $conv1d_X $conv1d_Y R0
```

Listing 3.20: Conv1d SRAM placement in floorplan.tcl

Standard cells are placed in the central region between the SRAM columns.

3.6.2 Area results

Table 3.5 shows the post-P&R area results from the OpenROAD final report.

Table 3.5: Post-P&R area results

Metric	Value
Die Area	26.01 mm^2 ($5100 \times 5100 \mu\text{m}$)
Core Area	21.80 mm^2
Total Active Area	5.87 mm^2
Core Utilization	26.93%
Std Cell Utilization	9.11%
user_domain (total)	$239,827 \mu\text{m}^2$
StdCell	$189,338 \mu\text{m}^2$
Macro (SRAM)	$50,489 \mu\text{m}^2$

3.6.3 Power results

Table 3.6 shows the power breakdown at 47.6 MHz (21 ns clock period) from the OpenROAD final report.

Table 3.6: Power breakdown by type

Type	Power (mW)	%
Internal	52.2	88.1%
Switching	7.0	11.8%
Leakage	0.03	<0.1%
Total	59.2	100%

Table 3.7: Power breakdown by category

Category	Power (mW)	%
Sequential	12.4	20.9%
Combinational	28.6	48.3%
Clock Network	16.0	27.0%
Pad	2.26	3.8%

3.6.4 Timing results

Table 3.8 summarizes the timing results.

Table 3.8: Timing results

Metric	Value
Clock Period	21 ns (47.6 MHz)
WNS (Setup)	0.00 ns
TNS (Setup)	0.00 ns
Worst Setup Slack	+3.91 ns (JTAG) / +4.88 ns (clk_sys)
Slack/Delay Ratio	15.05% (JTAG)
Setup Violations	0

Setup timing was achieved with good margin: the overall worst-case setup slack is +3.91 ns, which occurs on the JTAG clock domain (40 ns period, output path to `jtag_tdo_o`). On the 21 ns system clock domain, the worst setup slack is +4.88 ns (output path to `status_o`), corresponding to a data arrival time of 9.72 ns. There are 33 minor hold violations (worst -0.28 ns), which occur on SRAM interface paths. These are harmless and can be resolved with buffer insertion in a post-P&R optimization step.

3.6.5 Layout

Figure 3.10 shows the final layout of the Croc SoC after place and route. The 16 main SRAM banks are arranged in two columns (8 per side), the conv1d SRAM macro is visible at the bottom center, standard cells fill the central region, and the pad ring surrounds the perimeter.

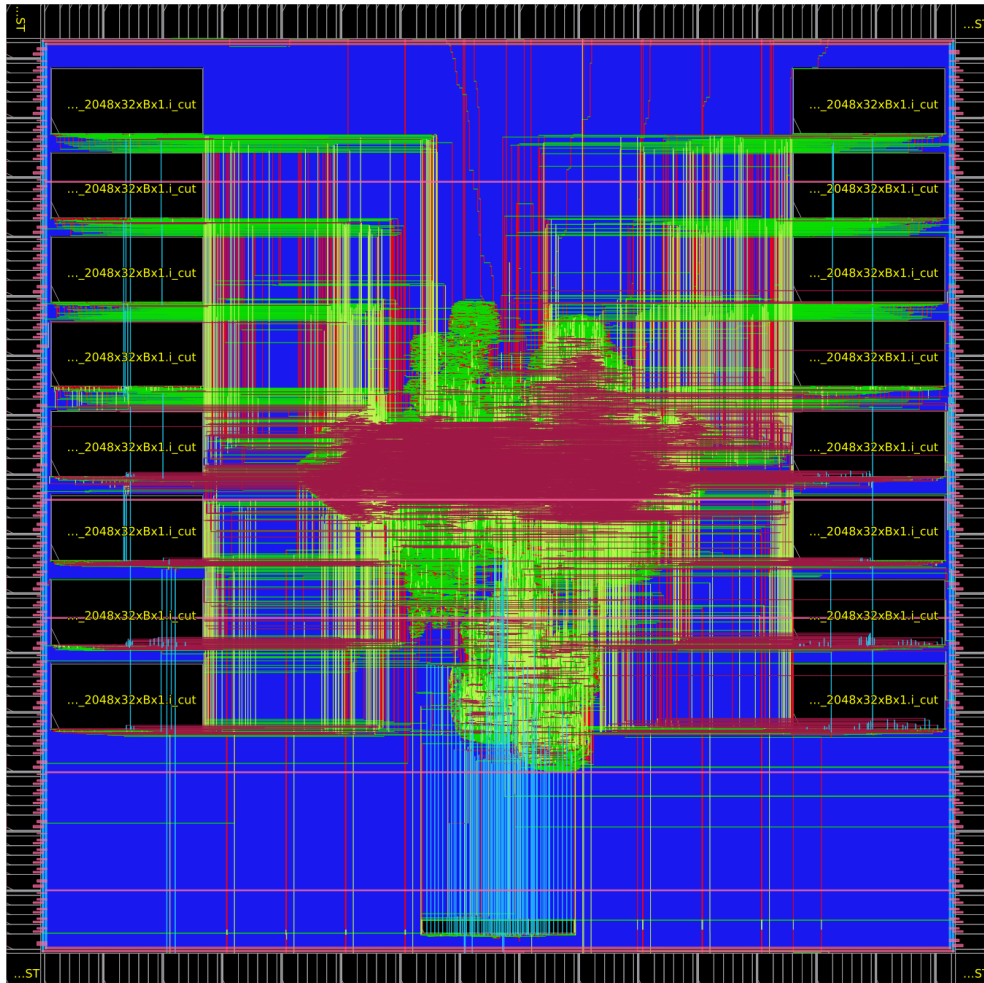


Figure 3.10: Final layout of the Croc SoC after place and route

3.6.6 Explanations and comments

The die utilization is approximately 27%, indicating significant unused area. As noted in the isa_lab3_2025_2026_part2.pdf, the die size could optionally be reduced significantly while still accommodating all macros and logic.

Timing closure was achieved with comfortable margin (+3.91 ns worst case JTAG slack, +4.88 ns worst case system clock slack), suggesting the design could operate at a higher frequency if needed.

Tables 3.6 and 3.7 show two independent breakdowns of the total 59.2 mW. Table 3.6 splits power by physical origin: internal power within cells dominates at 88.1%, switching of interconnect and load capacitances accounts for 11.8%, and leakage is negligible.

Table 3.7 splits the same total by circuit group. Combinational logic leads at 28.6 mW (48.3%), driven by the multiply and accumulate units in the conv1d datapath. The clock network consumes 16.0 mW (27.0%): although it has fewer nodes than the datapath, every clock buffer toggles on every cycle, and 4.72 mW of its 16.0 mW is switching power (making the clock network responsible for roughly two thirds of all switching power in the design). Sequential cells contribute 12.4 mW (20.9%) and pads 2.26 mW (3.8%), the latter almost entirely from switching as they drive off chip loads.

SRAM macros report 0 mW because the macro cell libraries lack power characterization data, so OpenROAD cannot estimate their contribution; in practice the SRAMs would add to the total.